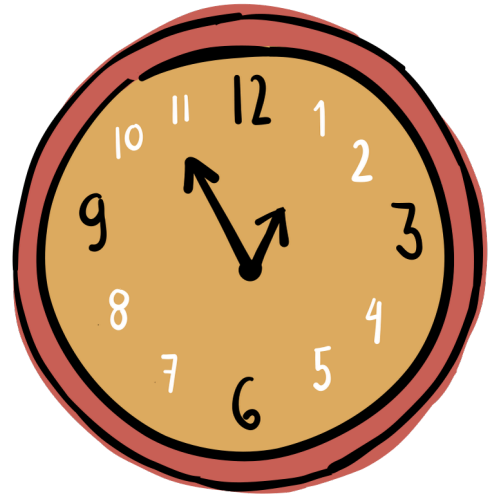
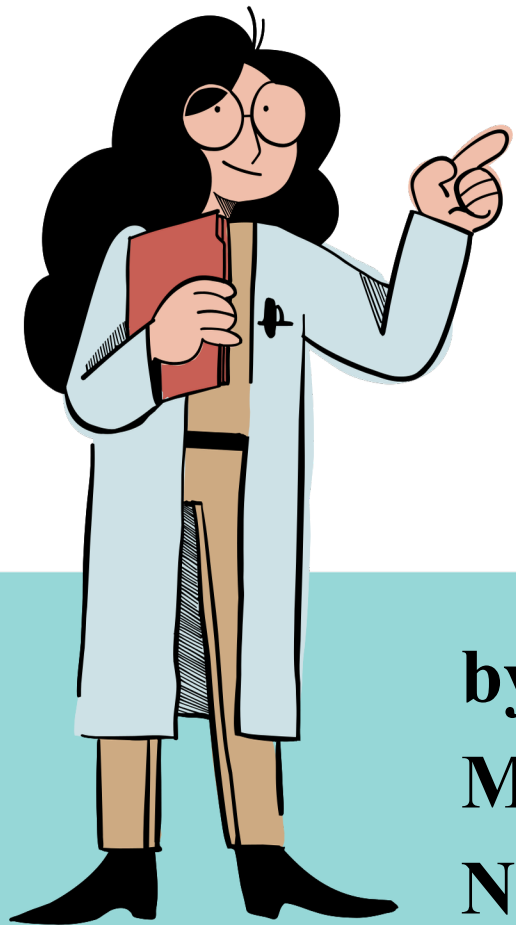




SYSTEM DESIGN HACKATHON

SYSCRAFTERS 2026



by

Mukilan MuthuKumar K -24AD136

Nithish K -24AD146

Rithik Regnesh AK -24AD176



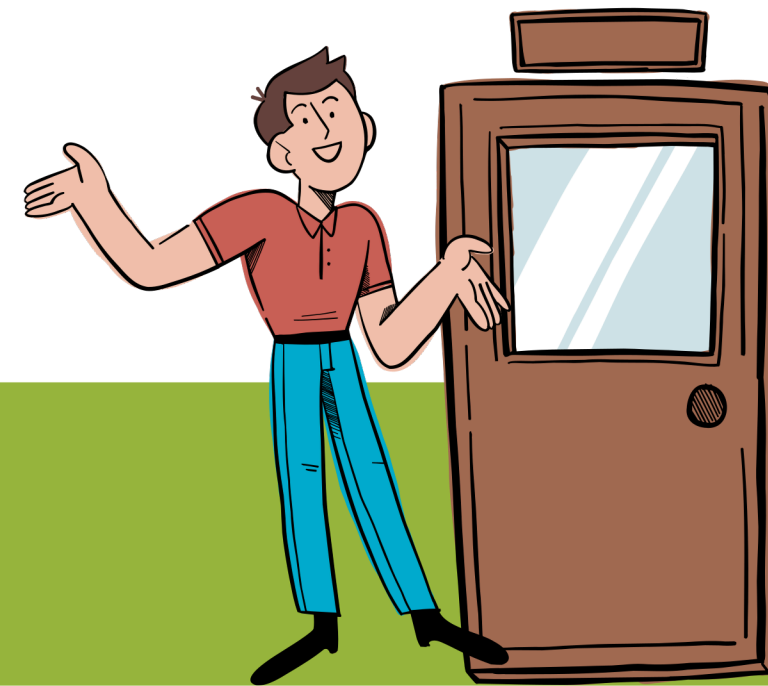
Problem & Business Requirements

Key Problems

- Demand Overload: Thousands compete for extremely limited stock.
- Race for Inventory: Concurrent requests can cause overselling.
- Duplicate Actions: Retries/double-clicks can trigger duplicate reservations or payments.
- Partial Failures: Payment may succeed while Order Service is unavailable.
- Stuck Inventory: Failed or expired transactions can lock stock unnecessarily.

Business Requirements

- Zero Overselling — sell only available inventory.
- High Concurrency — handle massive flash-sale traffic.
- Idempotent Transactions — one action, one reservation/payment.
- Reliable Recovery — no lost orders or inventory.
- Scalable & Observable — remain fast, fault-tolerant, and traceable.



SALESTORM:

Proposed Solution

Our Design Principle:

Control the request → Lock the inventory → Secure the payment → Guarantee the order.



Control the Crowd

- API Gateway + rate limiting absorb the traffic surge.
- Only eligible requests enter the purchase flow.

Reserve Before You Sell

- Inventory service performs an atomic reservation.
- Stock moves through Available → Reserved → Confirmed.

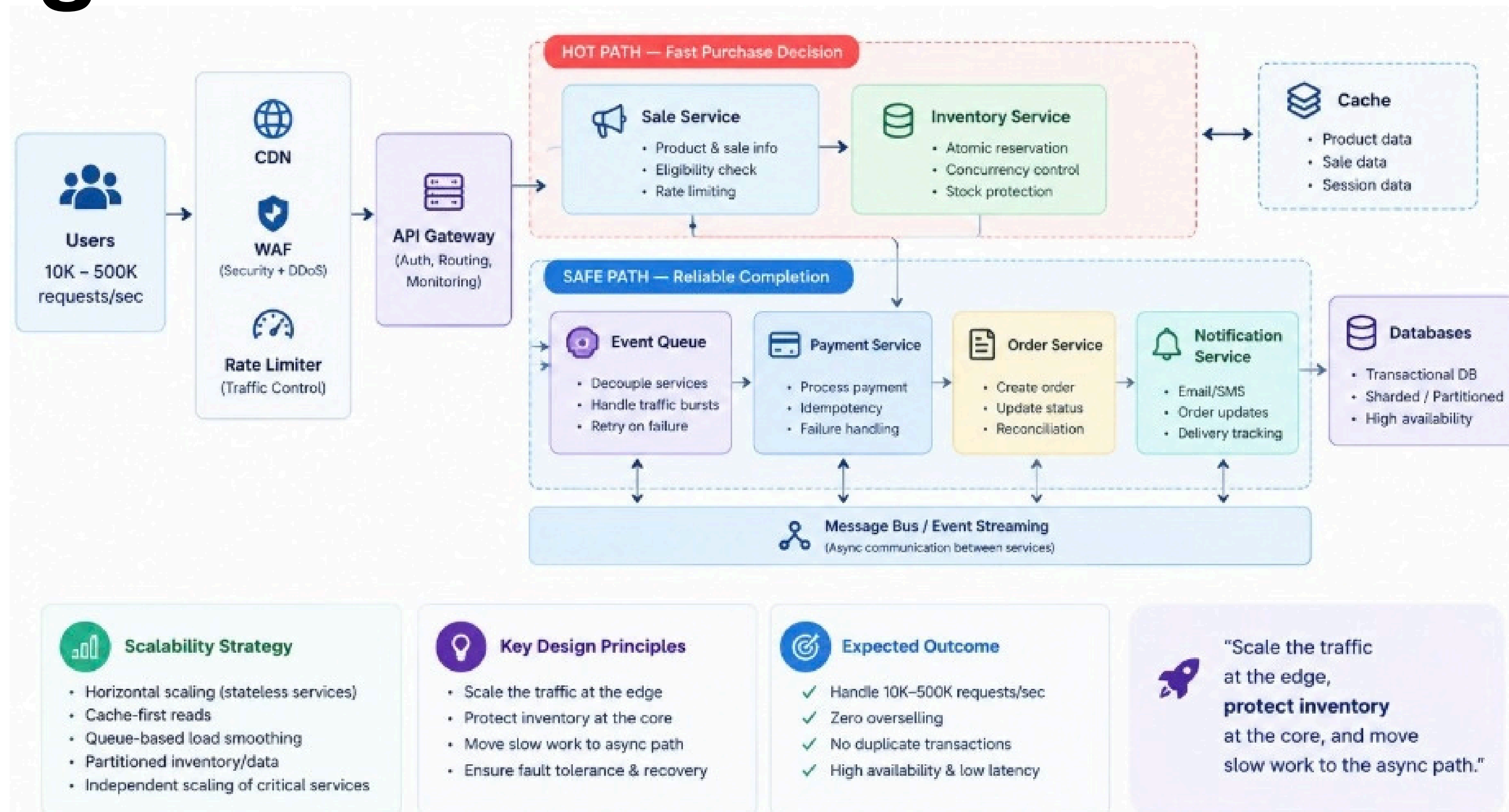
Make Every Action Safe

- Idempotency keys prevent duplicate reservations and payments.
- Expired/failed reservations automatically release stock.

Decouple Payment & Order

- Events/queues ensure payment success is not lost if Order Service fails.
- Retries + reconciliation complete interrupted transactions.

High-Level Architecture & Scalability



Inventory Reservation & Concurrency Control

“OneStockPool.ThousandsofContenders.OneConsistent Winner.”

Our Control Mechanism

- Atomic reservation ensures stock is claimed by only one successful request.
- Concurrency control protects the inventory record during competing updates.
- Idempotency key prevents the same customer from reserving twice.
- Reservation expiry automatically returns abandoned stock.

Reservation Lifecycle

AVAILABLE



RESERVED



PAYMENT PENDING



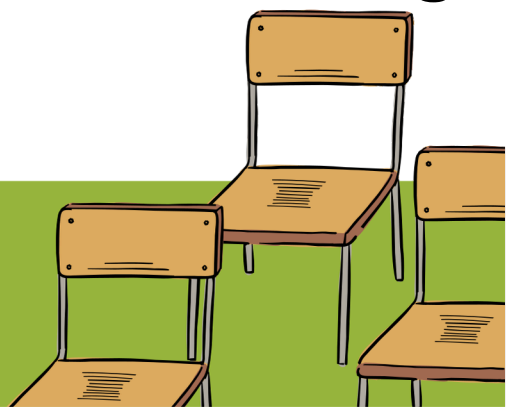
CONFIRMED / SOLD

Failure → RELEASE → AVAILABLE

Non-Negotiable Rule

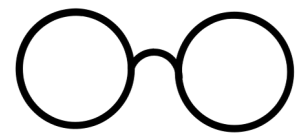
Successful Reservations $\leq$ Available Inventory

100 units → Maximum 100 successful reservations → Zero Overselling



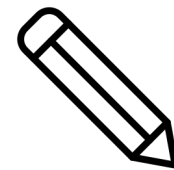
Purchase, Payment & Order Flow

One Purchase → One Payment → One Order



Purchase

- Validate request + Idempotency Key
- Confirm active reservation
- Create Payment Pending state



Payment

- Process payment against the reservation
- Success → publish Payment Success event
- Failure → release reservation + notify user
- Duplicate request → return existing payment result



Order

- Consume payment event asynchronously
- Create order only once using idempotency
- Confirm inventory → SOLD
- Order Service failure → queue event for retry

Flow

RESERVED → PAYMENT → PAYMENT SUCCESS → ORDER CREATED → SOLD

Failure Handling & Reliability

Failure	SALESTORM Response
Payment Failed	Release reservation → return stock
Payment Timeout	Safe retry / reconciliation → prevent double charge
Order Service Down	Persist event → retry → create order
Duplicate Message	Idempotent consumer → process once
Async Processing Fails	Retry → Dead-Letter Queue
External Service Unstable	Circuit Breaker → fail fast → recover



Data, APIs & Reliability

One Transaction. One Source of Truth.

DATA

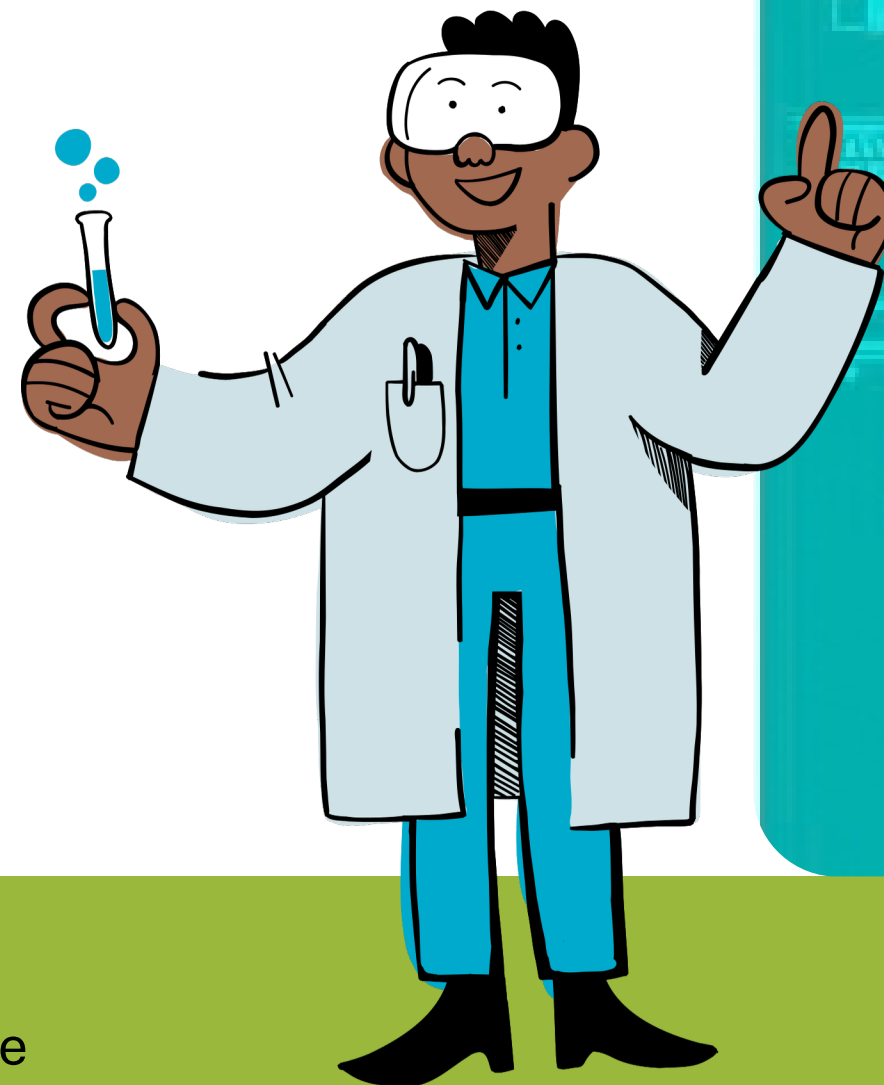
- Inventory tracks Available → Reserved → Sold
- Reservations store expiry + idempotency key
- Payment & Order maintain independent transaction states

APIs

- POST /reserve → POST /pay → POST /order
- Authentication + input validation
- Idempotency prevents duplicate business transactions

RELIABILITY

- Critical updates use transaction boundaries & constraints
- Events connect services without tight coupling
- Failed messages → Retry → DLQ → Reconciliation



Core Principle

“API trigger actions. Data protects state. Events preserve continuity.”

Scalability, Observability & Trade-Offs

01

Scalability

- Horizontal scaling for high traffic
- Load balancing + caching
- Queueing to handle traffic spikes
- Support up to 500K requests/sec

02

Observability

- Monitor latency, errors & traffic
- Track inventory and payment failures
- Distributed tracing for Payment → Order
- Alerts for critical failures

03

Trade-Offs

- Consistency over availability for inventory
- Synchronous for critical decisions
- Asynchronous for non-critical processing



Thank You

